

# DSA & Advanced DSA Roadmap

## Phase 0 — Problem Solving Foundation

### Learning Objectives

#### By the end of this phase, learners will be able to:

- Define what an algorithm is and describe its key properties (finiteness, definiteness, correctness).
- Analyze time and space complexity of simple code snippets and assign Big-O notation correctly.
- Distinguish between best-case, average-case, and worst-case complexity.
- Read and write basic recursive functions, identifying base cases and recursive cases.
- Trace (dry run) a piece of code by hand to predict its output and execution flow.
- Apply input/output optimization techniques relevant to competitive programming and coding platforms.

### Core Concepts

#### What is an Algorithm?

##### Concepts

- Definition: a finite, well-defined sequence of steps to solve a problem.
- Properties: finiteness, definiteness, input/output, effectiveness.
- Difference between an algorithm and a program.

##### Practice / Discussion Prompts

- Write the steps to find the largest number in a list, in plain English, before coding it.
- Discuss: can an algorithm have zero inputs? Zero outputs?

#### Time Complexity

##### Concepts

- Counting operations as a function of input size  $n$ .
- Growth rates: constant, logarithmic, linear, linearithmic, quadratic, exponential.
- Why we ignore constant factors and lower-order terms.

##### Practice / Discussion Prompts

- Given a loop, a nested loop, and a halving loop, identify each one's growth rate.
- Compare how  $O(n)$  and  $O(n^2)$  algorithms scale as  $n$  grows from 10 to 10,000.

#### Space Complexity

##### Concepts

- Auxiliary space vs total space.
- Space cost of recursion (call stack) vs iteration.
- In-place vs out-of-place algorithms.

### Practice / Discussion Prompts

- Compare the space complexity of an iterative factorial vs a recursive factorial.
- Identify which of two given sorting approaches uses  $O(1)$  extra space.

## Big O Notation

### Concepts

- Formal definition: upper bound on growth rate.
- Big-O, Big-Omega, and Big-Theta at a conceptual level.
- Rules for combining complexities (sum rule, product rule, dropping constants).

### Practice / Discussion Prompts

- Simplify given complexity expressions, e.g.  $3n^2 + 5n + 100 \rightarrow O(n^2)$ .
- Identify which term dominates as  $n$  approaches infinity for two algorithms compared side by side.

## Complexity Analysis

### Concepts

- Analyzing single loops, nested loops, and sequential blocks of code.
- Analyzing recursive functions using the recursion tree or recurrence relation.
- Common pitfalls: hidden loops inside library functions (e.g. sorting, string concatenation).

### Practice / Discussion Prompts

- Analyze a given function with three back-to-back loops and state its overall complexity.
- Analyze a recursive function that makes two recursive calls per invocation.

## Input & Output Optimization

### Concepts

- Buffered input/output vs unbuffered I/O.
- Fast I/O techniques relevant to competitive programming judges (e.g. avoiding repeated print calls in tight loops).
- Reading large inputs efficiently (bulk read vs line-by-line read).

### Practice / Discussion Prompts

- Identify why a correct solution might Time Limit Exceed (TLE) purely due to slow I/O.
- Rewrite a snippet that prints inside a loop to instead batch output and print once.

## Dry Run Techniques

### Concepts

- Hand-tracing variables, loop counters, and conditions step by step.
- Using a trace table to track variable state across iterations.
- Dry-running recursive calls using a call stack diagram.

### Practice / Discussion Prompts

- Dry run a given nested-loop snippet and produce its printed output by hand.
- Dry run a recursive function for a small input and draw the resulting call tree.

## Recursion Fundamentals

### Concepts

- Base case and recursive case.
- Call stack behavior: how frames are pushed and popped.
- Direct recursion vs indirect (mutual) recursion.
- Recursion vs iteration: when each is preferable.

### Practice / Discussion Prompts

- Write a recursive function to compute factorial, then convert it to an iterative version.
- Explain, using a stack diagram, why a recursive function without a base case causes a stack overflow.

## Phase Assessment

### Conceptual Questions

1. Given a code snippet with a single loop running  $n$  times containing constant-time work, derive its time complexity and justify it.
2. Given a snippet with two nested loops both running  $n$  times, derive its time complexity. Then modify the scenario so the inner loop runs only up to the outer loop's current index, and explain how the complexity changes.
3. Explain, in your own words, the difference between time complexity and space complexity, using a recursive Fibonacci function as the example.
4. Why does a recursive function require a base case? Describe what happens on the call stack if the base case is missing or unreachable.
5. Dry run the following pseudocode by hand and write out the final value of `total`: a loop from  $i = 1$  to 5, where total starts at 0 and  $total = total + i$  on each iteration. Show the trace table.
6. A program reads  $10^6$  integers one at a time and prints a running sum after every single integer ( $10^6$  print calls total). Explain why this could cause a Time Limit Exceeded verdict even though the underlying algorithm is  $O(n)$ , and describe a fix.
7. Rank the following complexities from fastest-growing to slowest-growing for large  $n$ :  $O(\log n)$ ,  $O(n^2)$ ,  $O(1)$ ,  $O(n \log n)$ ,  $O(2^n)$ ,  $O(n)$ .

### Introductory Coding Problems

- [Fibonacci Number — leetcode.com/problems/fibonacci-number](https://leetcode.com/problems/fibonacci-number)
- [Power of Two — leetcode.com/problems/power-of-two](https://leetcode.com/problems/power-of-two)
- [Reverse String — leetcode.com/problems/reverse-string](https://leetcode.com/problems/reverse-string)
- [Add Digits — leetcode.com/problems/add-digits](https://leetcode.com/problems/add-digits)
- [Pow\(x, n\) — leetcode.com/problems/powx-n](https://leetcode.com/problems/powx-n)

#### On scope

*For each problem above, students should submit two versions where applicable: a recursive solution and an iterative solution, then compare time and space complexity between the two. This directly reinforces the recursion-vs-iteration learning objective rather than just producing a passing submission.*

## Expected Outcome

### Students should be able to:

- Estimate the time and space complexity of a piece of code without running it.
- Read, write, and trace simple recursive functions, correctly identifying base cases.
- Explain why an algorithm that is correct can still fail on a judge due to inefficiency (time or I/O).
- Approach Phase 1 (Arrays) with the analytical vocabulary needed to reason about pattern efficiency.